Deep Learning in Practice

A Handbook for Building, Evaluating

and Reporting Projects

Deep Learning

MSc Artificial Intelligence Applications and Innovation

Imperial College London

November 2026

Contents

How to Use This Handbook 1

Quick Start 2

I Before You Train Anything (To Do) 4

1 Define the Project (To Do) 4

2 Understand and Split the Data (To Do) 4

II Building and Training (To Do) 5

3 Build an End-to-End Baseline (To Do) 5

4 Debug Training (To Do) 5

5 Improve Through Controlled Experiments (To Do) 6

III Evaluation and Reporting (To Do) 6

6 Evaluate and Analyse Errors (To Do) 6

7 Report and Hand Over (To Do) 7

A Project Checklist (To Do) 7

B Task Adaptation Cards (To Do) 7

How to Use This Handbook

Use this handbook while you plan, build and evaluate a deep learning project. It follows the decisions you need to make along the way: choosing a baseline, preparing data, investigating training problems and working out what the results support.

Who this is for

This handbook accompanies the Deep Learning course for Imperial's MSc Artificial Intelligence Applications and Innovation. It assumes basic Python and familiarity with tensors, neural networks, loss functions and backpropagation. The lecture notes explain the theory; the chapters here connect it to project work.

A small image-classification project runs through the main chapters. Keeping one example makes it easier to follow changes to the data, code and experiments. The workflow uses a single machine, with a small CPU test before longer runs. Appendix B covers adaptations for segmentation, text and time series.

Where to start

Begin with Quick Start to set up the environment and run a small test. Then read Chapters 1 and 2: define the task, choose a metric and decide how to split the data. Chapter 3 takes you through a complete baseline, from training and validation to saving a model and reloading it for prediction.

For a project already under way, Chapter 4 helps you investigate unexpected behaviour. Falling training loss alongside rising validation loss, for example, can indicate overfitting. Inspect the data, evaluation code and learning curves before choosing a change. Chapter 5 explains how to test that change through a controlled comparison. Record what you changed and why, including runs that did not help.

Read Chapters 6 and 7 early enough to plan your final evaluation and write-up. Keep the test set aside during development, and collect the comparisons and error examples you will need to explain your findings.

Working with the code

Examples use PyTorch and torchvision. Short listings show the part being discussed and may depend on earlier definitions. Read them with the explanation and expected output.

Before adapting an example, run a small version. Inspect a batch of inputs and labels, check the tensor shapes and examine a few predictions. Make one change at a time so that failures are easier to trace. Use validation results to judge model quality once these basic checks pass.

Using the checklists

Each chapter ends with a checklist and an output, such as a fixed data split or an experiment table. Use them to check your work before moving on; revisit the explanation if you cannot justify a choice. Appendix A brings the checks together for a final review. For assessed work, follow the assignment brief for submission requirements, permitted tools and disclosure of assistance.

Quick Start

Before starting a project, check that you can train a small model, save it and use it in a separate prediction run. This section uses generated images of vertical and horizontal stripes, so no dataset download is needed. The task is deliberately simple: its purpose is to check the installation and the training pipeline.

1. Set up an environment

Open a terminal in the handbook folder, where `examples/` is visible. The example has three source files: `quickstart.py` contains the code, `smoke.json` sets the data size and training options, and `requirements.txt` pins the package versions. All three are in `examples/`.

Use Python 3.12. In a macOS or Linux shell, run:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r examples/requirements.txt -i https://pypi.org/simple
python -c "import sys, torch; print(sys.version); print(torch.__version__)"
```

On Windows PowerShell, create the environment with `py -3.12 -m venv .venv` and activate it with `.venv\Scripts\Activate.ps1`, then run the two `python` commands. The printed versions should show Python 3.12 and PyTorch 2.14.1. Use the same environment whenever you return to the example. For platform-specific builds, consult the official PyTorch installation guide; the commands here run the model on CPU.

2. Run a small training job

The default configuration uses 64 training images, 32 separate validation images and three epochs. Each image has one channel and is 16×16 pixels. Run:

```
python examples/quickstart.py train --out runs/quickstart
```

A reference run produced the following output. Small numerical differences between systems are normal.

```
Device: cpu
Batch: images=(16, 1, 16, 16), labels=(16,)
epoch=1 train_loss=0.5334 val_loss=0.3632 val_accuracy=1.000
epoch=2 train_loss=0.2901 val_loss=0.1788 val_accuracy=1.000
epoch=3 train_loss=0.1331 val_loss=0.0762 val_accuracy=1.000
Reload check: passed
Saved run: runs/quickstart
```

The batch line describes 16 images and 16 labels. Loss should remain finite; the reload check confirms that restoring the saved parameters reproduces the model's output. The high accuracy reflects how easy the stripe task is. Performance on real images needs the data splits and evaluation procedure covered in later chapters.

3. Reload the model and make a prediction

Run prediction as a separate command. This starts a new Python process and uses the saved model and a saved validation input:

```
python examples/quickstart.py predict --out runs/quickstart
```

With the supplied configuration, the output should include `Prediction: vertical`. The class probabilities are written to `runs/quickstart/prediction.csv`.

The loading code in `examples/quickstart.py` rebuilds the network and restores its parameters:

```
model = make_model(config["image_size"])
state = torch.load(
    output / "model.pt", map_location="cpu", weights_only=True
)
model.load_state_dict(state)
model.eval()
```

Here `make_model` is defined in the same file. The prediction function then runs the forward pass inside `torch.inference_mode()`. Keeping model construction and loading together helps avoid a mismatch between the saved parameters and the network. See the PyTorch save/load tutorial for more detail.

4. Check what the run saved

Open `runs/quickstart/` and inspect these files:

File	What it records
`config.json`	Settings used for this run.
`metrics.csv`	Training loss, validation loss and accuracy per epoch.
`model.pt`	Saved model parameters.
`input.pt`	One validation image for the prediction check.
`environment.json`	Python/PyTorch versions, platform, device and timing.
`prediction.csv`	Predicted class and probabilities from the reload run.

The reference run used Python 3.12.7 and PyTorch 2.14.1 on a macOS ARM64 CPU with one thread. Training and saving took less than a second; installation and Python startup take additional time. Your timing is recorded in `environment.json`. Keep these files together so that the model can be traced back to its settings and results.

If a command fails, begin with the error message:

- `No module named torch`: check that the environment is active and install packages using that environment's `python -m pip`.
- A missing script or configuration file: check the working directory. A missing checkpoint: complete the training command first.
- An existing output directory: choose a new `-out` path for both commands. Each run keeps its own files.

Before moving on: check that training logged three epochs, the reload check passed, and a separate prediction run created `prediction.csv`. Then continue to Chapter 1 to plan your own project.

Part I
Before You Train Anything (To Do)

1 Define the Project (To Do)

Budget: 3 pages. Goal: define a feasible question and success criteria.

- Translate the application into inputs, outputs, intended users and a measurable task; decide whether deep learning is justified.
- Choose a primary metric, a success criterion and practical constraints before training; explain why accuracy can mislead.
- Define a meaningful simple baseline and, where appropriate, a non-DL baseline; use the closest prior work to justify choices.
- Set data, time and compute budgets, milestones and a fallback plan; identify data permissions and foreseeable harms early.

Code: introduce the shared configuration for task, metric, baseline and run budget.
Output/checkpoint: a one-page project plan with an explicit baseline, success criterion and stopping point.

2 Understand and Split the Data (To Do)

Budget: 4 pages. Goal: build a trustworthy input and evaluation pipeline.

- Inspect samples, labels, class counts, duplicates and missing metadata; record provenance, licence and privacy constraints.
- Match random, group or temporal splits to the intended use. Define validation for development and hold the test set aside.
- Split before fitting preprocessing; estimate statistics from training data only and apply augmentation only during training.
- Save sample IDs, labels and split assignments in a versioned manifest; check overlap and inspect a transformed batch.

Code: manifest creation, split assertions and batch inspection with `Dataset`/`DataLoader`.
Output/checkpoint: a fixed split manifest, data summary and verified examples of the tensors and labels the model will receive.

Part II
Building and Training (To Do)

3 Build an End-to-End Baseline (To Do)

Budget: 6 pages. Goal: run and understand the complete project loop.

- Evaluate the simple reference baseline from Chapter 1 and choose a modest neural model; explain when pretrained weights are a useful starting point.
- Connect batch shapes, logits, targets and loss; explain gradients, optimiser steps and the metric without re-deriving backpropagation.
- Write training and validation loops with the correct model modes and gradient handling; first check that a tiny subset can be fitted.
- Record configuration, seed, code revision, dataset and split versions, environment, and training and validation metrics from the first run.
- Save a checkpoint selected using validation data, reload it and predict on a new input with the same preprocessing and class mapping.

Code: linked excerpts from data loading, model setup, `train`, `evaluate`, checkpointing and `predict`.

Output/checkpoint: a baseline run with interpretable curves, saved metadata and a checkpoint that can be reloaded for prediction.

4 Debug Training (To Do)

Budget: 5 pages. Goal: locate failures before spending more compute.

- Follow a diagnostic order: inspect a batch, check predictions and loss, fit a tiny subset, then inspect full training/validation curves.
- Use symptom–check–action tables for flat loss, NaNs, shape/device errors, suspiciously good validation scores and poor generalisation.
- Work through one deliberately broken example, explaining the evidence that identifies the bug and confirms the fix.
- Diagnose single-device memory and throughput limits before changing batch size, using mixed precision or tuning data loading.

Code: assertions, a tiny-subset run and minimal diagnostics for tensors, gradients, timing and memory.

Output/checkpoint: a documented diagnosis and a verified fix; unexplained behaviour is resolved before larger experiments begin.

5 Improve Through Controlled Experiments (To Do)

Budget: 4 pages. Goal: make improvements that can be explained.

- Turn error analysis into a hypothesis; compare feature extraction, fine-tuning or training from scratch where relevant to the task.
- Prioritise a small search over learning rate, batch size and model capacity; introduce regularisation and schedulers as needed.
- Compare runs with the same data splits, metric and a stated compute budget; design ablations to isolate a specific contribution.
- Extend the existing run records into a comparison table with validation results, runtime, compute use and reasons for each change.
- Use validation-based early stopping and a fixed search budget; record unsuccessful experiments and keep the test set untouched.

Code: configuration overrides and aggregation of run logs into a comparison table; no tracking service is required.
Output/checkpoint: an experiment table and a justified final configuration, with the selection rule recorded before test evaluation.

Part III

Evaluation and Reporting (To Do)

6 Evaluate and Analyse Errors (To Do)

Budget: 5 pages. Goal: establish what the results support.

- Freeze model choices, preprocessing and any decision thresholds using validation data before evaluating the held-out test set.
- Interpret the primary metric alongside class-wise results, confusion patterns and inspected errors; assess relevant subgroups.
- Distinguish variation across training seeds from uncertainty due to a finite test sample. Use a small example; explain when confidence intervals are meaningful and respect grouping when resampling.
- Present controlled ablations and negative results; relate gains to compute cost, practical usefulness and the original success criterion.
- State distribution-shift risks, failure cases and limits of use; keep detailed statistical testing in optional resources.

Code: a fixed evaluation entry point, error inspection and a simple uncertainty example with its assumptions stated.
Output/checkpoint: a final results table, representative errors and a clear account of uncertainty and limitations.

7 Report and Hand Over (To Do)

Budget: 3 pages. Goal: make the work understandable and reusable.

- Structure the report around question, data, methods, experiments, findings and limitations; connect each claim to supporting evidence.
- Show concise figures and tables with meaningful axes, units, captions and stated uncertainty; explain negative results honestly.
- Provide a README, pinned dependencies, data-access instructions, saved configuration and checkpoint, and exact commands for evaluation and prediction.
- Verify the handover from a clean environment. Check permissions before sharing data or weights; report compute and follow the module's current rules for attribution and AI-assistance disclosure.
- Prepare a brief demonstration using new inputs and include a visible failure case and an explanation of appropriate use.

Code: the documented evaluation and prediction commands, with example inputs and expected outputs.

Output/checkpoint: a concise report and a handover package that another student can use to reproduce evaluation and run predictions.

A Project Checklist (To Do)

Budget: 1 page. A compact index of the chapter checks, with references to the relevant explanations:

- **Start**: question, baseline, metric, feasibility and permissions.
- **Before training**: inspected data, appropriate splits, preprocessing checks and complete run records.
- **Before final evaluation**: resolved training failures, justified comparisons and frozen selection decisions.
- **Before handover**: supported claims, limitations, reproducible evaluation, working predictions and required disclosures.

B Task Adaptation Cards (To Do)

Budget: 3 pages total. Three one-page cards comparing each task with the main example. Each card states inputs/outputs, a useful baseline, loss/metric choices, split rules and the most important pitfalls:

- Segmentation: masks, spatial alignment, class imbalance and image/subject-level independence.
- Text classification: tokenisation, padding/masking, pretrained models and duplicate/source leakage.
- Time-series forecasting: forecast horizon, temporal validation, simple forecast baselines and future-information leakage.

Generative modelling and reinforcement learning need additional workflows and evaluation guidance; include separate optional repository guides only if they are within the module's project scope. Platform setup notes and longer task examples also belong in the repository. Recurring lab questions should improve the relevant chapter's troubleshooting table.